Lab 3 Herencia, Análisis del caso TecniHogar

CC2008 Programación Orientada a Objetos, Universidad del Valle de Guatemala, Ciclo 2, 2026.

Requisitos funcionales

Información que el sistema debe conservar

De cada orden de servicio el sistema conserva el código único, el nombre del cliente, la dirección, las horas estimadas de trabajo que pueden tener decimales, la zona que puede ser metropolitana o periférica, el estado actual, la cantidad de visitas realizadas y la jornada a la que está asignada en caso de que lo esté. Cada categoría conserva además sus datos propios. El diagnóstico no agrega datos. La instalación conserva la cantidad de puntos a instalar. La instalación eléctrica certificada conserva los puntos y el valor declarado de los materiales. El mantenimiento de aire acondicionado conserva la capacidad del equipo en BTU.

De cada cuadrilla el sistema conserva el código único, la capacidad máxima de horas de trabajo por jornada, la indicación de si está certificada en refrigeración y la jornada que tiene sin cerrar, si la tiene. De cada jornada conserva un número que la identifica, la cuadrilla elegida, la lista de órdenes asignadas, el estado de la jornada y el resultado registrado de cada orden.

De los ingresos el sistema conserva cada movimiento por separado, con su tipo que puede ser registro o visita adicional, su monto, la categoría y el código de la orden que lo originó. Los totales no se guardan como números sueltos, se calculan a partir de los movimientos, de modo que no pueden quedar desfasados y solo cambian cuando se agrega un movimiento real.

Reglas de cobro y tiempos de atención

El diagnóstico cuesta Q150 en zona metropolitana y Q225 en zona periférica, sin importar las horas. Sus horas estimadas no pueden superar 1.5. Se atiende en 1 día hábil en zona metropolitana y en 2 días hábiles en zona periférica, y permite hasta 3 visitas.

La instalación cuesta la tarifa base de la zona, que es Q100 en zona metropolitana y Q175 en zona periférica, más Q90 por cada hora facturable o fracción. Las horas por puntos son la cantidad de puntos por 0.75 y las horas facturables son el mayor valor entre las horas estimadas y las horas por puntos. Como se cobra cada hora o fracción, las horas facturables se redondean hacia arriba a la siguiente hora entera antes de multiplicar por Q90. Con 2.5 horas estimadas y 5 puntos en zona metropolitana las horas por puntos son 3.75, se facturan 4 horas y el costo es Q100 más Q360, es decir Q460. Se atiende en 3 días hábiles en zona metropolitana y en 5 días hábiles en zona periférica, y permite hasta 3 visitas.

La instalación eléctrica certificada se calcula igual que una instalación y suma Q120 por el certificado y una garantía equivalente al 5 por ciento del valor declarado de los materiales. Con los datos del ejemplo anterior y materiales por Q2,000 cuesta Q460 más Q120 más Q100, es decir Q680. Se atiende en dos días hábiles más que una instalación normal de la misma zona, o sea 5 días en zona metropolitana y 7 días en zona periférica, y permite hasta 2 visitas.

El mantenimiento de aire acondicionado solo se acepta en zona metropolitana, con una capacidad entre 9,000 y 60,000 BTU. Cuesta la tarifa base de la zona, que es Q100, más Q110 por cada hora estimada y un cargo fijo de Q80 por gas refrigerante. Con 2 horas estimadas cuesta Q100 más Q220 más Q80, es decir Q400. Se atiende siempre en 2 días hábiles y permite una sola visita.

Cada visita adicional a la primera genera un cargo de Q40 que se registra como ingreso en el momento en que la jornada inicia, y no cuando se asigna la orden. Cuando una orden agota sus visitas queda cancelada y no puede asignarse de nuevo.

Supuestos adoptados

Se interpreta que por cada hora o fracción significa redondear hacia arriba las horas facturables, tal como lo confirma el ejemplo de 3.75 horas que se factura como 4. En el mantenimiento de aire acondicionado se cobran Q110 por cada hora estimada sin redondear, porque el enunciado no menciona fracciones para esa categoría. Una jornada puede llegar justo a la capacidad de la cuadrilla, porque la condición dice que la suma no supera la capacidad. El valor declarado de los materiales debe ser mayor que cero. La cotización valida los datos pero no revisa si el código ya existe, porque no registra nada, y esa revisión se hace al registrar. El costo de una orden se calcula a partir de sus datos, que no cambian después de registrada, por lo que el monto cobrado al registrar siempre coincide con el costo consultado después.

Operaciones que el sistema debe permitir y sus condiciones de validez

Cotizar una orden. Recibe la categoría y los datos del servicio y muestra el costo y el tiempo estimado de atención. Es válida cuando el código, el cliente y la dirección no están vacíos, las horas estimadas son mayores que cero, la zona es una de las dos permitidas y se cumplen los datos propios de la categoría, que son horas máximas de 1.5 en el diagnóstico, puntos enteros positivos en la instalación, materiales positivos en la instalación eléctrica y zona metropolitana con BTU entre 9,000 y 60,000 en el mantenimiento de aire. No registra la orden, no cambia ningún estado y no altera los ingresos.

Registrar una orden. Requiere las mismas condiciones que la cotización y además que el código no exista ya entre las órdenes registradas. La orden queda pendiente, con cero visitas, y se registra un ingreso por registro igual al costo completo de la orden.

Registrar una cuadrilla. Requiere un código no vacío y no repetido, una capacidad de horas mayor que cero y la indicación de si está certificada en refrigeración. La cuadrilla queda libre.

Crear una jornada. Requiere que la cuadrilla exista y que no tenga otra jornada sin cerrar. La jornada queda en preparación y la cuadrilla queda ocupada.

Asignar una orden a una jornada. Requiere que la jornada exista y esté en preparación, que la orden exista y esté pendiente, lo que implica que no está asignada a otra jornada ni cancelada ni completada, que la orden sea compatible con la cuadrilla y que la suma de las horas estimadas de las órdenes ya asignadas más las de la nueva no supere la capacidad de la cuadrilla. Para la capacidad se usan las horas estimadas y nunca las horas facturables. La orden pasa a asignada.

Retirar una orden de una jornada. Requiere que la jornada esté en preparación y que la orden pertenezca a ella. La orden vuelve a quedar pendiente y los ingresos no cambian.

Iniciar una jornada. Requiere que la jornada esté en preparación y que tenga al menos una orden. La jornada pasa a estar en curso, todas sus órdenes pasan a estar en atención, a cada orden se le suma una visita realizada y por cada orden que ya lleva más de una visita se registra un ingreso por visita adicional de Q40.

Registrar el resultado de una orden. Requiere que la jornada esté en curso, que la orden pertenezca a la jornada y que todavía esté en atención, es decir que no tenga resultado registrado, y que el resultado sea completada o visita fallida. Una orden completada queda completada. Una orden con visita fallida queda cancelada si ya agotó las visitas permitidas por su categoría, y en caso contrario vuelve a quedar pendiente. Cuando todas las órdenes de la jornada tienen resultado, la jornada queda cerrada y la cuadrilla queda libre.

Consultar órdenes y cuadrillas. Solo muestra información y no cambia ningún estado. Muestra de cada orden el código, la categoría, el cliente, la zona, las horas estimadas, los datos propios, el costo, el tiempo de atención, las visitas realizadas y permitidas y el estado. Muestra de cada cuadrilla el código, la capacidad, la certificación y si está libre u ocupada.

Generar reportes. Muestra la cantidad de órdenes por categoría y por estado, el ingreso total separado en ingresos por registro y por visitas adicionales, el ingreso por categoría y las jornadas que no están cerradas con su cuadrilla, sus horas asignadas y su capacidad disponible. Los reportes solo leen datos.

Regla general. Toda operación inválida se rechaza con un mensaje claro y sin modificar el estado previo ni los ingresos. Para lograrlo, cada operación verifica todas sus condiciones antes de cambiar cualquier dato. Las entradas con formato incorrecto, como texto donde se espera un número, se rechazan en el menú sin terminar el programa.

Decisiones que dependen de la categoría y la clase que las toma

El costo depende de la categoría. Lo calcula cada clase concreta en su método calcularCosto, porque cada una es la dueña de los datos que intervienen, como los puntos, los materiales o el equipo en BTU. La instalación eléctrica certificada reutiliza el cálculo de la instalación y le suma lo propio.

El tiempo estimado de atención depende de la categoría y de la zona. Lo decide cada clase concreta en tiempoAtencionDias. La instalación eléctrica certificada lo expresa como el de la instalación más dos días, de modo que si cambia el de la instalación cambia también el de ella.

El número de visitas permitidas depende de la categoría. Lo decide cada clase concreta en visitasMaximas, y la clase base usa ese valor para decidir si una visita fallida cancela la orden o la devuelve a pendiente. Así la política de cancelación está escrita una sola vez y solo el número cambia.

La compatibilidad con la cuadrilla depende de la categoría. La decide la orden en admiteCuadrilla. La clase base responde que sí a cualquier cuadrilla y el mantenimiento de aire acondicionado sobrescribe el método para exigir la certificación en refrigeración. Se coloca en

la orden y no en la jornada porque la jornada no debe conocer las categorías, y si lo hiciera tendría que verificar el tipo del objeto y modificarse con cada categoría nueva.

La validez de los datos propios depende de la categoría. Cada constructor valida lo suyo después de que el constructor de la clase base valida lo común. El diagnóstico valida el máximo de 1.5 horas, la instalación valida los puntos, la instalación eléctrica valida los materiales y el mantenimiento valida la zona metropolitana y el rango de BTU. Así una orden inválida nunca llega a existir.

El nombre de la categoría depende de la categoría. Lo devuelve cada clase concreta en getCategoria, y los reportes lo usan para agrupar órdenes e ingresos sin verificar el tipo del objeto. Es importante que la instalación eléctrica certificada sobrescriba este método, porque de lo contrario los reportes la contarían como una instalación normal.

La descripción de los datos propios depende de la categoría. Cada clase concreta aporta su parte en detalleEspecifico, y la clase base arma con ella la descripción completa de la orden para las consultas.

Hay decisiones que no dependen de la categoría y por eso no se sobrescriben. La verificación de capacidad la toma la jornada, porque solo usa horas estimadas, que toda orden tiene. La regla de una sola jornada abierta por cuadrilla la toma la cuadrilla. Los cambios de estado y el conteo de visitas los toma la clase base de las órdenes. El monto del cargo por visita adicional es igual para todas las categorías, por lo que vive en la clase base, y el momento de registrarlo, que es al iniciar la jornada, lo decide la jornada. El cálculo de los totales lo toma el registro de ingresos.

Estados de las órdenes y de las jornadas

Estados de una orden

Una orden nace como pendiente cuando se registra. Desde pendiente pasa a asignada cuando se asigna a una jornada en preparación. Desde asignada vuelve a pendiente cuando se retira de la jornada, y pasa a en atención cuando la jornada inicia, momento en que se suma la visita y se cobra el cargo adicional si corresponde. Desde en atención pasa a completada cuando se registra el resultado completada. Si se registra visita fallida pasa a cancelada cuando las visitas realizadas ya alcanzan las permitidas por su categoría, y en caso contrario vuelve a pendiente y puede asignarse a otra jornada. Los estados completada y cancelada son finales.

Una orden solo puede asignarse si está pendiente, solo puede retirarse si está asignada, y solo admite resultado si está en atención. Cualquier otra combinación se rechaza con un mensaje y sin cambios.

Estados de una jornada

Una jornada nace en preparación cuando se crea. En ese estado se pueden asignar y retirar órdenes. Pasa a en curso cuando se inicia, lo cual exige que tenga al menos una orden. En curso solo se pueden registrar resultados, y no se pueden asignar ni retirar órdenes. Pasa a cerrada cuando todas sus órdenes tienen resultado registrado, y en ese momento la cuadrilla

queda libre. Cerrada es un estado final. Una jornada en preparación o en curso mantiene ocupada a su cuadrilla, por lo que una cuadrilla nunca tiene más de una jornada sin cerrar.

Clases necesarias y sus responsabilidades

Todas las clases se agrupan en un solo paquete. Los atributos son privados salvo que se indique otra cosa, y las colecciones se entregan hacia afuera como listas no modificables para que nadie cambie el estado interno sin pasar por los métodos de la clase. Los métodos que solo deben ser usados por otras clases del modelo, como los que cambian el estado de una orden, tienen visibilidad de paquete.

Enumeraciones

Zona tiene los valores metropolitana y periférica. Guarda un atributo privado y final tarifaBase de tipo double, con valor 100 para la metropolitana y 175 para la periférica, que sirve como tarifa base de la instalación y del mantenimiento. Su constructor es privado y recibe la tarifa, y tiene el método público getTarifaBase que no recibe parámetros y devuelve double. La tarifa del diagnóstico no se guarda aquí porque es una regla propia de esa categoría.

EstadoOrden tiene los valores pendiente, asignada, en atención, completada y cancelada. EstadoJornada tiene los valores en preparación, en curso y cerrada. ResultadoVisita tiene los valores completada y fallida. TipoIngreso tiene los valores registro y visita adicional.

Clase abstracta OrdenServicio

Representa lo común a todas las órdenes y es abstracta porque no existe un servicio sin categoría. Sus atributos privados son codigo de tipo String que identifica la orden, cliente de tipo String con el nombre del cliente, direccion de tipo String con el lugar de la visita, horasEstimadas de tipo double con las horas de trabajo, zona de tipo Zona, estado de tipo EstadoOrden, visitasRealizadas de tipo int que cuenta las visitas iniciadas y jornadaAsignada de tipo Jornada que es la jornada actual o nulo cuando no tiene. Tiene además la constante estática privada cargoVisitaAdicional de tipo double con valor 40.

Su constructor es protegido y recibe codigo, cliente, direccion, horasEstimadas y zona. Valida que los textos no estén vacíos, que las horas sean mayores que cero y que la zona no sea nula, usando la clase Validador, y guarda esos valores. Los valores iniciales de estado, visitasRealizadas y jornadaAsignada no se reciben como parámetros, el constructor los fija en pendiente, cero y nulo, porque toda orden nueva parte de ahí.

Sus métodos abstractos son calcularCosto, público, sin parámetros, devuelve double con el monto a cobrar. tiempoAtencionDias, público, sin parámetros, devuelve int con los días hábiles estimados. visitasMaximas, público, sin parámetros, devuelve int con las visitas permitidas. getCategoria, público, sin parámetros, devuelve String con el nombre de la categoría. detalleEspecifico, protegido, sin parámetros, devuelve String con la descripción de los datos propios de la categoría.

Su método admiteCuadrilla es público, recibe una Cuadrilla y devuelve boolean. Tiene una implementación por defecto que devuelve verdadero y está pensado para ser sobrescrito por las categorías con restricciones.

Sus métodos de cambio de estado tienen visibilidad de paquete. asignarA recibe una Jornada, exige que la orden esté pendiente, guarda la jornada y pasa a asignada, sin devolver nada. liberar exige que esté asignada, borra la jornada y vuelve a pendiente, sin devolver nada. iniciarVisita no recibe parámetros, exige que esté asignada, pasa a en atención, suma una visita y devuelve un double con el cargo que se debe registrar, que es cero para la primera visita y cargoVisitaAdicional para las siguientes. registrarResultado recibe un ResultadoVisita, exige que esté en atención, borra la jornada y deja la orden completada, cancelada o pendiente según el resultado y según si visitasRealizadas alcanzó el valor de visitasMaximas, sin devolver nada. Todos lanzan una OperacionInvalidaException si el estado no lo permite y no cambian nada antes de lanzarla.

Tiene métodos públicos de consulta getCodigo, getCliente, getDireccion, getHorasEstimadas, getZona, getEstado, getVisitasRealizadas y getJornadaAsignada, sin parámetros y con el tipo de su atributo como resultado. Sobrescribe toString de Object para devolver una descripción de una línea que combina los datos comunes con detalleEspecifico, el costo y el tiempo de atención.

Clase Diagnostico

Hereda de OrdenServicio. No tiene atributos propios. Tiene las constantes estáticas privadas horasMaximas con valor 1.5, tarifaMetropolitana con valor 150 y tarifaPeriferica con valor 225. Su constructor público recibe los mismos cinco datos comunes, los pasa al constructor de la clase base con super y después valida que las horas no superen 1.5. Sobrescribe calcularCosto para devolver la tarifa fija según la zona, tiempoAtencionDias para devolver 1 o 2 según la zona, visitasMaximas para devolver 3, getCategoria para devolver Diagnóstico y detalleEspecifico para indicar que no tiene datos adicionales.

Clase Instalacion

Hereda de OrdenServicio. Su atributo privado puntos es de tipo int y guarda los puntos a instalar. Tiene las constantes estáticas privadas horasPorPunto con valor 0.75 y tarifaPorHora con valor 90. Su constructor público recibe los cinco datos comunes y los puntos, llama a super con los datos comunes y valida que los puntos sean mayores que cero.

Tiene el método público horasPorPuntos, sin parámetros, que devuelve double con puntos por 0.75, y el método público horasFacturables, sin parámetros, que devuelve double con el mayor valor entre horasEstimadas y horasPorPuntos. Sobrescribe calcularCosto para devolver la tarifa base de la zona más la tarifa por hora multiplicada por las horas facturables redondeadas hacia arriba. Sobrescribe tiempoAtencionDias para devolver 3 o 5 según la zona, visitasMaximas para devolver 3, getCategoria para devolver Instalación y detalleEspecifico para mostrar los puntos y las horas facturables. Tiene el método público getPuntos.

Clase InstalacionElectricaCertificada

Hereda de Instalacion. Su atributo privado valorMateriales es de tipo double y guarda el valor declarado de los materiales. Tiene las constantes estáticas privadas cargoCertificado con valor 120 y porcentajeGarantia con valor 0.05. Su constructor público recibe los datos comunes, los

puntos y el valor de los materiales, llama a super con los datos de la instalación y valida que el valor de los materiales sea mayor que cero.

Tiene el método público calcularGarantia, sin parámetros, que devuelve double con el 5 por ciento del valor declarado. Sobrescribe calcularCosto para devolver el costo de la instalación obtenido con super más el cargo del certificado más la garantía. Sobrescribe tiempoAtencionDias para devolver el valor de la instalación más 2, visitasMaximas para devolver 2, getCategoria para devolver Instalación eléctrica certificada y detalleEspecifico para agregar los materiales a lo que ya describe la instalación. Tiene el método público getValorMateriales.

Clase MantenimientoAire

Hereda de OrdenServicio. Su atributo privado capacidadBtu es de tipo int y guarda la capacidad del equipo. Tiene las constantes estáticas privadas btuMinimo con valor 9000, btuMaximo con valor 60000, tarifaPorHora con valor 110 y cargoGas con valor 80. Su constructor público recibe los datos comunes y la capacidad en BTU, llama a super y después valida que la zona sea metropolitana y que la capacidad esté entre el mínimo y el máximo.

Sobrescribe calcularCosto para devolver la tarifa base de la zona más la tarifa por hora por las horas estimadas más el cargo del gas. Sobrescribe tiempoAtencionDias para devolver siempre 2, visitasMaximas para devolver 1, getCategoria para devolver Mantenimiento de aire acondicionado y detalleEspecifico para mostrar la capacidad. Sobrescribe admiteCuadrilla para devolver la certificación en refrigeración de la cuadrilla recibida. Tiene el método público getCapacidadBtu.

Clase Cuadrilla

Representa a un equipo de técnicos. Sus atributos privados son codigo de tipo String, capacidadHoras de tipo double con el máximo de horas por jornada, certificadaRefrigeracion de tipo boolean y jornadaActual de tipo Jornada, que es nulo cuando está libre. Su constructor público recibe el código, la capacidad y la certificación, valida que el código no esté vacío y que la capacidad sea mayor que cero, y fija jornadaActual en nulo.

Sus métodos públicos son getCodigo, getCapacidadHoras, isCertificadaRefrigeracion y getJornadaActual, que devuelven el valor de su atributo, estaLibre que devuelve boolean y toString sobrescrito. Sus métodos de paquete son ocupar, que recibe una Jornada, exige que la cuadrilla esté libre y la guarda, y liberar, que deja la cuadrilla libre. Ninguno de los dos devuelve nada y ocupar lanza una OperacionInvalidaException si ya hay una jornada sin cerrar.

Clase Jornada

Agrupa las órdenes que una cuadrilla atenderá. Sus atributos privados son numero de tipo int que la identifica, cuadrilla de tipo Cuadrilla, ordenes de tipo lista de OrdenServicio, resultados de tipo mapa de OrdenServicio a ResultadoVisita con los resultados ya registrados, y estado de tipo EstadoJornada. Su constructor público recibe el número y la cuadrilla, pide a la cuadrilla que se ocupe, lo cual falla si ya tiene una jornada sin cerrar, crea la lista y el mapa vacíos y fija el estado en preparación.

El método público asignar recibe una OrdenServicio y no devuelve nada. Verifica que la jornada esté en preparación, que la orden esté pendiente, que la orden acepte a la cuadrilla mediante admiteCuadrilla y que la suma de horas más las de la orden no supere la capacidad, y solo entonces agrega la orden y le pide asignarA. Aquí se valida la compatibilidad con la cuadrilla, sin verificar ningún tipo de orden.

El método público retirar recibe una OrdenServicio y no devuelve nada. Verifica que la jornada esté en preparación y que la orden pertenezca a ella, y solo entonces la quita de la lista y le pide liberar.

El método público iniciar recibe el RegistroIngresos y no devuelve nada. Verifica primero que la jornada esté en preparación y que no esté vacía. Después pasa a en curso y, para cada orden, llama a iniciarVisita y registra en los ingresos el cargo recibido cuando es mayor que cero, con tipo visita adicional. Aquí se registra el cargo por visita adicional.

El método público registrarResultado recibe una OrdenServicio y un ResultadoVisita y no devuelve nada. Verifica que la jornada esté en curso, que la orden pertenezca a ella y que no tenga ya resultado, le pide registrarResultado, guarda el resultado en el mapa y, si todas las órdenes ya tienen resultado, pasa a cerrada y libera a la cuadrilla.

Los métodos públicos horasAsignadas y capacidadDisponible no reciben parámetros y devuelven double. El primero suma las horas estimadas de las órdenes de la jornada y el segundo resta esa suma de la capacidad de la cuadrilla. También tiene getNumero, getCuadrilla, getEstado, getOrdenes que devuelve una lista no modificable, estaCerrada que devuelve boolean y toString sobrescrito.

Clases Ingreso y RegistroIngresos

Ingreso representa un movimiento de dinero y es inmutable. Sus atributos privados y finales son tipo de tipo TipoIngreso, monto de tipo double, categoria de tipo String y codigoOrden de tipo String. Su constructor público recibe los cuatro valores y valida que el monto sea mayor que cero. Solo tiene métodos de consulta.

RegistroIngresos es la única clase que guarda los ingresos. Su atributo privado movimientos es una lista de Ingreso, vacía al construirse. El método público registrar recibe tipo, monto, categoría y código de orden, crea el Ingreso y lo agrega, sin devolver nada. Los métodos públicos total y totalPorTipo, este último con un TipoIngreso como parámetro, devuelven double. El método público totalPorCategoria no recibe parámetros y devuelve un mapa de nombre de categoría a monto. Ningún método modifica ni elimina movimientos, por eso las cotizaciones, los retiros y las operaciones rechazadas no pueden alterar los ingresos, ya que nunca llaman a registrar.

Clase TecniHogar

Es la clase de servicio que coordina el modelo y que usa el menú. Sus atributos privados son ordenes de tipo mapa de código a OrdenServicio, cuadrillas de tipo mapa de código a Cuadrilla, jornadas de tipo lista de Jornada, ingresos de tipo RegistroIngresos y siguienteNumeroJornada de tipo int. Su constructor público no recibe parámetros, crea las colecciones vacías, el registro de ingresos y fija el número de jornada en 1.

Sus métodos públicos son cotizar, que recibe una OrdenServicio y devuelve un String con el costo y el tiempo sin guardar nada, registrarOrden, que recibe una OrdenServicio, verifica que el código no exista, la guarda y registra el ingreso por registro con su costo, y registrarCuadrilla, que recibe una Cuadrilla y verifica que el código no exista. Tiene también crearJornada, que recibe el código de la cuadrilla y devuelve la Jornada creada, asignarOrden, retirarOrden y registrarResultado, que reciben el número de jornada, el código de la orden y, en el último, el resultado, e iniciarJornada, que recibe el número de jornada. Estos métodos buscan los objetos por su identificador, lanzan una OperacionInvalidaException si no existen y delegan en la jornada.

Sus métodos de consulta y reporte son buscarOrden, buscarCuadrilla, listarOrdenes, listarCuadrillas, conteoPorCategoriaYEstado que devuelve un mapa de categoría a mapa de estado a cantidad, getIngresos, y jornadasAbiertas que devuelve la lista de jornadas que no están cerradas. Los reportes usan solo getCategoria y getEstado de las órdenes, nunca el tipo del objeto.

Clases de apoyo y clase principal

OperacionInvalidaException hereda de RuntimeException y su constructor recibe el mensaje que se mostrará al usuario. Se lanza en toda operación inválida y el menú la atrapa para mostrar el mensaje y continuar. Validador es una clase final con constructor privado y métodos estáticos públicos como textoNoVacio y mayorQueCero, que lanzan la excepción, y evita repetir validaciones en cada clase.

MenuConsola tiene como atributos privados un Scanner y el objeto TecniHogar. Su método público ejecutar muestra el menú hasta que el usuario elige salir, y sus métodos privados leen enteros, decimales, textos y zonas atrapando los errores de formato para volver a pedir el dato sin terminar el programa. Es el único lugar que conoce los constructores de las categorías concretas, para pedir los datos y crear la orden. DatosIniciales tiene el método estático público cargar, que recibe TecniHogar y registra los datos de arranque sin ingresos, y para ello se usa un método interno que agrega órdenes sin cobrar, de modo que los ingresos inician en cero. Incluye al menos dos órdenes pendientes de cada categoría, entre ellas una instalación de 2 horas estimadas y 5 puntos cuyas horas por puntos, 3.75, superan las estimadas, y al menos tres cuadrillas, una certificada en refrigeración y otra no certificada. La clase Main contiene el método main, que crea TecniHogar, carga los datos iniciales y ejecuta el menú.

Relaciones entre las clases

Diagnostico, Instalacion y MantenimientoAire heredan de OrdenServicio, e InstalacionElectricaCertificada hereda de Instalacion. Una Jornada se asocia con exactamente una Cuadrilla, y una Cuadrilla tiene cero o una Jornada sin cerrar. Una Jornada contiene cero o más órdenes y una orden pertenece a cero o una jornada a la vez. TecniHogar contiene cero o más órdenes, cuadrillas y jornadas, y exactamente un RegistroIngresos, el cual contiene cero o más Ingreso. Cada orden usa una Zona.

Cómo reciben sus valores iniciales los objetos

El menú o los datos iniciales llaman al constructor de una clase concreta con todos los datos. Ese constructor pasa los datos comunes al de su clase padre con super, y el de la clase base valida y guarda lo común y fija los valores iniciales. La instalación eléctrica certificada llama al constructor de Instalacion, que a su vez llama al de OrdenServicio, de modo que cada nivel valida solo lo que le corresponde. Si cualquier validación falla se lanza una excepción y el objeto no llega a crearse, por lo que nunca existe una orden a medias ni se guarda en el sistema. Las cuadrillas, las jornadas y los ingresos siguen la misma idea, validan en su constructor y fijan sus estados iniciales sin recibirlos como parámetros.

Justificación de la relación entre la instalación y la instalación eléctrica certificada

Se propone que InstalacionElectricaCertificada herede de Instalacion porque cumple la relación es un. Una instalación eléctrica certificada es una instalación, que además requiere certificado y garantía. Comparte todos los datos de la instalación, que son los puntos, las horas por puntos y las horas facturables, y comparte sus reglas, porque el enunciado dice que se calcula igual y que se le suman cargos. La herencia permite escribir esas reglas una sola vez, y la subclase solo agrega lo que cambia, que es el costo con super más el certificado y la garantía, el tiempo con super más dos días, el límite de 2 visitas y el dato de los materiales. Si el precio o el tiempo de la instalación cambian, la eléctrica cambia con ella sin tocar su código. Además cumple la sustitución, porque en cualquier lugar donde se espera una instalación se puede usar una eléctrica sin que nada se rompa.

Una primera alternativa es que InstalacionElectricaCertificada herede directamente de OrdenServicio. Se perdería la reutilización, porque habría que copiar los puntos, las horas por puntos, las horas facturables, el redondeo y las tarifas. Esa duplicación de reglas es peligrosa porque si se corrige una y se olvida la otra, la empresa cobraría distinto una misma instalación. Se ganaría independencia entre las dos clases, pero ese beneficio no compensa el riesgo.

Una segunda alternativa es no crear la clase y agregar a Instalacion un atributo booleano certificada y los materiales. Se ahorraría una clase, pero toda instalación normal cargaría atributos que no usa, y los métodos de costo, tiempo y visitas tendrían que preguntar si es certificada con una condición, que es justo la verificación que el polimorfismo busca evitar. Cada variante nueva agregaría más condiciones a la misma clase, y los reportes tendrían que distinguir la categoría con una condición adicional.

Una tercera alternativa es la composición, es decir que la instalación eléctrica contenga un objeto Instalacion y le delegue el cálculo. Evita acoplar las clases con herencia y permitiría cambiar la instalación base en tiempo de ejecución, pero obliga a escribir métodos que solo reenvían llamadas y, sobre todo, la eléctrica dejaría de ser una OrdenServicio por sí misma, o tendría que repetir toda la interfaz de la orden. Como aquí la relación es de especialización y no cambia en ejecución, la herencia es más simple y más natural.

La herencia tiene un costo, que es el acoplamiento entre padre e hija. Para controlarlo, la instalación expone horasFacturables y calcularCosto como parte de su contrato público y los atributos permanecen privados, de modo que la subclase usa a la clase padre solo mediante sus métodos.

Impacto de agregar una nueva categoría

Para agregar una categoría nueva, como instalación de paneles solares, se crea una sola clase nueva que hereda de OrdenServicio, o de una categoría existente si es una especialización de ella. Esa clase tiene sus atributos propios, un constructor que llama a super y valida sus datos, e implementa calcularCosto, tiempoAtencionDias, visitasMaximas, getCategoria y detalleEspecifico. Si tiene una restricción de cuadrilla, sobrescribe admiteCuadrilla con su regla.

Si la restricción necesita un dato que las cuadrillas todavía no tienen, por ejemplo una certificación en energía solar, hay que agregar ese atributo a Cuadrilla y a su registro. Es el único caso en que una clase existente cambia por razones del modelo, y el cambio es agregar un dato, no cambiar reglas.

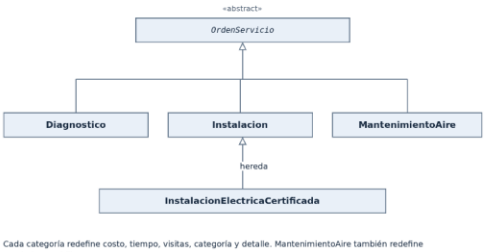
No cambian OrdenServicio, Jornada, RegistroIngresos ni TecniHogar. Las jornadas no cambian porque validan la compatibilidad llamando a admiteCuadrilla, validan la capacidad con las horas estimadas, que toda orden tiene, y cierran según resultados registrados, sin conocer la categoría. Los reportes no cambian porque agrupan por getCategoria y por estado, así que la categoría nueva aparece sola en los conteos y en los ingresos. El cargo por visita adicional y la cancelación tampoco cambian porque dependen de visitasMaximas, que la clase nueva define.

Lo único adicional que sí hay que tocar es la interfaz de consola, para agregar una opción que pida los datos propios de la nueva categoría y llame a su constructor, y opcionalmente DatosIniciales para incluir órdenes de ejemplo. Esa es una consecuencia de que el menú es el único lugar que conoce los constructores concretos, y la lógica de negocio queda intacta.

TecniHogar · Diagrama de clases UML

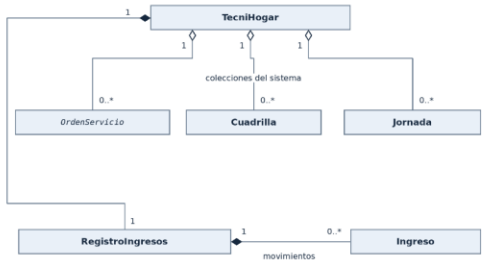
Lab 3 · Herencia · CC2008 · Diseño consistente con el enunciado y el análisis adjuntos

01 Herencia de los servicios



Cada categoría redefine costo, tiempo, visitas, categoría y detalle. MantenimientoAire también redefine admiteCuadrilla.

02 Catálogos e ingresos



03 Asignación e historial



Las referencias actuales son subconjuntos del historial. Una orden tiene como máximo una jornada activa; una cuadrilla, como máximo una jornada sin cerrar.

04 Atributos, constructores y operaciones

Las vistas superiores y los compartimentos siguientes representan las mismas clases. Tipos de colección de Java; visibilidad explícita.

+ público ~ privado # protegido ~ paquete Cursiva: abstracto Subrayado: estático «override»: método sobrescrito {readOnly}: final

OrdenServicio

«abstract»

- codigo: String
- cliente: String
- direccion: String
- horasEstimadas: double
- zona: Zona
- estado: EstadoOrden = PENDIENTE
- visitasRealizadas: int = 0
- jornadaAsignada: Jornada [0..1] = null
- cargoVisitaAdicional: double = 40 {readOnly}

OrdenServicio(codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: Zona)
+ *calcularCosto(): double*
+ *tiempoAtencionDias(): int*
+ *visitasMaximas(): int*
+ *getCategoria(): String*
detalleEspecifico(): String
+ admiteCuadrilla(cuadrilla: Cuadrilla): boolean
~ asignarA(jornada: Jornada): void
~ liberar(): void
~ iniciarVisita(): double
~ registrarResultado(resultado: ResultadoVisita): void
+ getCodigo(): String
+ getCliente(): String
+ getDireccion(): String
+ getHorasEstimadas(): double
+ getZona(): Zona
+ getEstado(): EstadoOrden
+ getVisitasRealizadas(): int
+ getJornadaAsignada(): Jornada [0..1]
«override» + toString(): String

Diagnostico

- horasMaximas: double = 1.5 {readOnly}
- tarifaMetropolitana: double = 150 {readOnly}
- tarifaPeriferica: double = 225 {readOnly}

+ Diagnostico(codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: Zona)
«override» + calcularCosto(): double
«override» + tiempoAtencionDias(): int
«override» + visitasMaximas(): int
«override» + getCategoria(): String
«override» # detalleEspecifico(): String

MantenimientoAire

- capacidadBtu: int
- btuMinimo: int = 9000 {readOnly}
- btuMaximo: int = 60000 {readOnly}
- tarifaPorHora: double = 110 {readOnly}
- cargoGas: double = 80 {readOnly}

+ MantenimientoAire(codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: Zona, capacidadBtu: int)
«override» + calcularCosto(): double
«override» + tiempoAtencionDias(): int
«override» + visitasMaximas(): int
«override» + getCategoria(): String
«override» # detalleEspecifico(): String
«override» + admiteCuadrilla(cuadrilla: Cuadrilla): boolean
+ getCapacidadBtu(): int

Instalacion

- puntos: int
- horasPorPunto: double = 0.75 {readOnly}
- tarifaPorHora: double = 90 {readOnly}

+ Instalacion(codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: Zona, puntos: int)
+ horasPorPuntos(): double
+ horasFacturables(): double
«override» + calcularCosto(): double
«override» + tiempoAtencionDias(): int
«override» + visitasMaximas(): int
«override» + getCategoria(): String
«override» # detalleEspecifico(): String
+ getPuntos(): int

InstalacionElectricaCertificada

- valorMateriales: double
- cargoCertificado: double = 120 {readOnly}
- porcentajeGarantia: double = 0.05 {readOnly}

+ InstalacionElectricaCertificada(codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: Zona, puntos: int, valorMateriales: double)
+ calcularGarantia(): double
«override» + calcularCosto(): double
«override» + tiempoAtencionDias(): int
«override» + visitasMaximas(): int
«override» + getCategoria(): String
«override» # detalleEspecifico(): String
+ getValorMateriales(): double

RegistroIngresos

- movimientos: List<Ingreso>

+ RegistroIngresos()
+ registrar(tipo: TipoIngreso, monto: double, categoria: String, codigoOrden: String): void
+ total(): double
+ totalPorTipo(tipo: TipoIngreso): double
+ totalPorCategoria(): Map<String, Double>

Ingreso

«inmutable»

- tipo: TipoIngreso {readOnly}
- monto: double {readOnly}
- categoria: String {readOnly}
- codigoOrden: String {readOnly}

+ Ingreso(tipo: TipoIngreso, monto: double, categoria: String, codigoOrden: String)
+ getTipo(): TipoIngreso
+ getMonto(): double
+ getCategoria(): String
+ getCodigoOrden(): String

TecniHogar

- ordenes: Map<String, OrdenServicio>
- cuadrillas: Map<String, Cuadrilla>
- jornadas: List<Jornada>
- ingresos: RegistroIngresos
- siguienteNumeroJornada: int = 1

+ TecniHogar()
+ cotizar(orden: OrdenServicio): String
+ registrarOrden(orden: OrdenServicio): void
+ registrarCuadrilla(cuadrilla: Cuadrilla): void
+ crearJornada(codigoCuadrilla: String): Jornada
+ asignarOrden(numeroJornada: int, codigoOrden: String): void
+ retirarOrden(numeroJornada: int, codigoOrden: String): void
+ iniciarJornada(numeroJornada: int): void
+ registrarResultado(numeroJornada: int, codigoOrden: String, resultado: ResultadoVisita): void
+ buscarOrden(codigo: String): OrdenServicio
+ buscarCuadrilla(codigo: String): Cuadrilla
+ listarOrdenes(): List<OrdenServicio>
+ listarCuadrillas(): List<Cuadrilla>
+ conteoPorCategoriaYEstado(): Map<String, Map<EstadoOrden, Integer>>
+ getIngresos(): RegistroIngresos
+ jornadasAbiertas(): List<Jornada>
~ cargarOrdenInicial(orden: OrdenServicio): void

Jornada

- numero: int
- cuadrilla: Cuadrilla
- ordenes: List<OrdenServicio>
- resultados: Map<OrdenServicio, ResultadoVisita>
- estado: EstadoJornada = EN_PREPARACION

+ Jornada(numero: int, cuadrilla: Cuadrilla)
+ asignar(orden: OrdenServicio): void
+ retirar(orden: OrdenServicio): void
+ iniciar(ingresos: RegistroIngresos): void
+ registrarResultado(orden: OrdenServicio, resultado: ResultadoVisita): void
+ horasAsignadas(): double
+ capacidadDisponible(): double
+ getNumero(): int
+ getCuadrilla(): Cuadrilla
+ getEstado(): EstadoJornada
+ getOrdenes(): List<OrdenServicio>
+ estaCerrada(): boolean
«override» + toString(): String

Cuadrilla

- codigo: String
- capacidadHoras: double
- certificadaRefrigeracion: boolean
- jornadaActual: Jornada [0..1] = null

+ Cuadrilla(codigo: String, capacidadHoras: double, certificadaRefrigeracion: boolean)
+ getCodigo(): String
+ getCapacidadHoras(): double
+ isCertificadaRefrigeracion(): boolean
+ getJornadaActual(): Jornada [0..1]
+ estaLibre(): boolean
~ ocupar(jornada: Jornada): void
~ liberar(): void
«override» + toString(): String

MenuConsola

- scanner: Scanner
- sistema: TecniHogar

+ MenuConsola(scanner: Scanner, sistema: TecniHogar)
+ ejecutar(): void
- leerEntero(mensaje: String): int
- leerDecimal(mensaje: String): double
- leerTexto(mensaje: String): String
- leerZona(mensaje: String): Zona
- crearOrden(): OrdenServicio

Main

—

- Main()
+ main(args: String[]): void

DatosIniciales

—

- DatosIniciales()
+ cargar(sistema: TecniHogar): void

Validador

«utility» {final»

—

- Validador()
+ textoNoVacio(valor: String, campo: String): void
+ mayorQueCero(valor: double, campo: String): void
+ mayorQueCero(valor: int, campo: String): void

OperacionInvalidaException

—

+ OperacionInvalidaException(mensaje: String)

RuntimeException

«externa» java.lang

05 Enumeraciones

Zona

«enumeration»

METROPOLITANA(100)
PERIFERICA(175)
- tarifaBase: double {readOnly}

- Zona(tarifaBase: double)
+ getTarifaBase(): double

EstadoOrden

«enumeration»

PENDIENTE
ASIGNADA
EN_ATENCION
COMPLETADA
CANCELADA

—

EstadoJornada

«enumeration»

EN_PREPARACION
EN_CURSO
CERRADA

—

ResultadoVisita

«enumeration»

COMPLETADA
FALLIDA

—

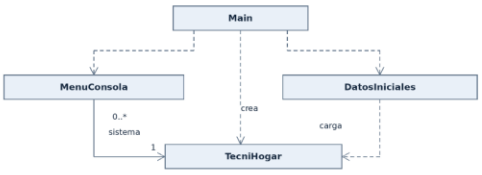
TipoIngreso

«enumeration»

REGISTRO
VISITA_ADICIONAL

—

06 Arranque y consola



07 Reglas de colaboración

Jornada.asignar valida orden.admiteCuadrilla(cuadrilla) y usa horas estimadas para la capacidad. No consulta el tipo concreto.

Jornada.iniciar llama a orden.iniciarVisita(). La orden incrementa las visitas y devuelve Q0 o Q40; la jornada registra el cargo solo si es mayor que cero.

Jornada.registrarResultado conserva el historial. Cuando todos los resultados están registrados, cierra la jornada y libera la cuadrilla.

MenuConsola y DatosIniciales crean las categorías concretas; Jornada usa RegistroIngresos. Las validaciones usan Validador y lanzan OperacionInvalidaException.

08 Restricciones y notación

Constructores: eléctrica → Instalacion → OrdenServicio, mediante super(...). Cada nivel valida sus datos antes de aceptar el objeto.

Visita fallida: CANCELADA si visitasRealizadas >= visitasMaximas(); en otro caso, PENDIENTE. Un resultado COMPLETADA mantiene ese estado incluso en la última visita.

TecniHogar valida códigos únicos. Cotizar, retirar y rechazar operaciones no registra ingresos. cargarOrdenInicial agrega las órdenes de ejemplo sin cobrar.

Las listas expuestas son no modificables. Los reportes agrupan por getCategoria() y getEstado(); los totales se calculan desde los movimientos.

Triángulo vacío: herencia. Rombo vacío: agregación. Rombo lleno: composición. Línea simple: asociación. Flecha discontinua: dependencia.